**Journal of Statistical Software** MMMMMM YYYY, Volume VV, Issue II.
doi:10.18637/jss.vVV.iII Submitted: DD MMMM YYYY; Accepted: DD MMMM YYYY.

# lepidoptera: A Domain-Agnostic Engine for Causal Simulation with Mixed Variable Types, Vine Copula Dependence, and Counterfactual Propagation

Danilo A. Sarti
Universidade Estadual de Campinas

**Abstract**

We introduce **lepidoptera**, an open-source R and Python package providing a modular, domain-agnostic engine for simulation from structural causal models (SCMs) with heterogeneous variable types. The package makes four contributions not available in combination in any existing software. First, a *typed mechanism* architecture separates the causal graph from the generating rules at each node, exposing each rule as an inspectable, serialisable `"Mechanism"` object that can be replaced in isolation for any intervention or sensitivity analysis. Second, a *pre-built mechanism library* covers all nine parent-type / child-type combinations (continuous, categorical, ordinal × continuous, categorical, ordinal), including linear Gaussian, multinomial logit, ordered logit, group-effect, and categorical transition models, removing the need for users to implement type dispatch from scratch. Third, a *vine copula mechanism* learns the full joint dependence structure from observed data via pair-copula constructions and the Rosenblatt conditional-simulation algorithm, supporting over twenty copula families from **rvinecopulib** (Nagler and Vatter, 2022). Fourth, *partial propagation* operationalises individual-level counterfactual queries by freezing the random draws of unaffected nodes and replaying only the downstream consequences of an intervention, cleanly separating the ITE estimand from the population-level ATE without requiring additional model assumptions. An AI-native extension layer allows any pre-fitted predictive model — random forests, gradient-boosted trees, neural networks — to serve as a generative mechanism. Simulation studies validate type-combination coverage, copula family fidelity, and counterfactual consistency. Real-data illustrations span clinical trial simulation, hospital emergency-ward digital twinning, and epidemiological mediation analysis. **lepidoptera** is available under the MIT licence at `https://github.com/daniloasarti/lepidoptera`.

**Keywords:** structural causal model, directed acyclic graph, mixed variable types, vine copula, counterfactual simulation, partial propagation, R, Python.

**Keywords:** structural causal model, directed acyclic graph, mixed variable types, vine copula, counterfactual simulation, partial propagation, R, Python.

## Contents

**1 Introduction** **4**
1.1 Motivation . . . . . 4
1.2 Overview of contributions . . . . . 4

1.3 Paper organisation . . . . . 5

**2 Related Work** **5**
2.1 Causal simulation in R . . . . . 5
2.2 Causal simulation in Python . . . . . 6
2.3 Counterfactual simulation . . . . . 6
2.4 Synthetic data and copula models . . . . . 6
2.5 Summary gap analysis . . . . . 7

**3 The lepidoptera Framework** **7**
3.1 Variable types and domains . . . . . 7
3.2 Mechanisms . . . . . 7
3.3 Systems . . . . . 8
3.4 Interventions . . . . . 8

**4 Mechanism Library** **8**
4.1 Pre-built factories . . . . . 8
4.2 Code examples . . . . . 9

**5 Simulation and Propagation Algorithms** **11**
5.1 Population simulation . . . . . 11
5.2 Full-resampling propagation . . . . . 11

**6 Counterfactual Propagation** **12**
6.1 Formal treatment . . . . . 12
6.2 Comparison of propagation modes . . . . . 12
6.3 Code example: counterfactual query . . . . . 13

**7 Vine Copula Mechanism** **13**
7.1 Motivation and scope . . . . . 13
7.2 Pair-copula constructions . . . . . 14
7.3 Rosenblatt conditional simulation . . . . . 14
7.4 Copula families . . . . . 14
7.5 Code example: copula mechanism . . . . . 15
7.6 Nominal variable handling . . . . . 16

**8 AI-Native Extension Layer** **16**
8.1 `learned_mechanism()`: fitting from observed data . . . . . 16
8.2 `model_mechanism()`: wrapping external models . . . . . 17
8.3 `llm_mechanism()`: language-model-generated features . . . . . 17

**9 Simulation Studies** **18**
9.1 Study design . . . . . 18
9.2 Type-combination coverage . . . . . 18
9.3 Copula family fidelity . . . . . 18
9.4 Counterfactual consistency . . . . . 19

**10 Real-Data Applications** **19**
10.1 Application 1: Clinical trial simulation with realistic confounding . . . . . 19
10.2 Application 2: Emergency-ward digital twin with partial propagation . . . . . 20
10.3 Application 3: Epidemiological mediation with natural direct and indirect effects 21

**11 Discussion** **22**
11.1 Summary of contributions . . . . . 22
11.2 Comparison with the concurrent **simDAG** . . . . . 23
11.3 Limitations . . . . . 23
11.4 Future directions . . . . . 23

**12 Computational Details** **23**

**A Full Type-Combination Simulation Tables** **27**

**B Copula Families: Extended Results** **27**

**C Complete R API Reference** **27**
C.1 Core constructors . . . . . 27
C.2 Simulation and propagation . . . . . 27
C.3 Mechanism library . . . . . 27
C.4 Serialisation . . . . . 28

# 1 Introduction

## 1.1 Motivation

Consider three scenarios that regularly arise in applied statistics.

**Example 1.1** (Clinical trial planning)**.** *A biostatistician designs a Phase II oncology trial. The patient population includes continuous covariates (age, tumour size, biomarker level), an ordinal baseline performance score (ECOG 0–4), and a categorical treatment group. The primary endpoint is a binary indicator of response, confounded by performance score. The analyst wants to (i) generate a realistic synthetic population that honours the observed joint distribution of these heterogeneous covariates, (ii) apply an intervention that fixes the treatment assignment for all patients (*population policy*), and (iii) ask, for each patient who actually failed, what their outcome* would have been *under a different dosing regimen (*individual counterfactual*).*

**Example 1.2** (Emergency-ward digital twin)**.** *A hospital data scientist builds a simulation engine to stress-test staffing protocols. The model includes continuous inter-arrival times, ordinal triage categories (Priority 1–3), continuous length-of-stay, and a categorical discharge destination. She wishes to intervene on triage protocols and measure the downstream effect on boarding times for individual patients, not just the population average.*

**Example 1.3** (Epidemiological mediation)**.** *An epidemiologist wants to decompose the effect of income on cardiovascular disease risk into a direct path and a path mediated through dietary quality and smoking status. Income is continuous; smoking is categorical; diet is continuous; CVD risk is binary. Estimating natural direct and indirect effects requires simulating counterfactual outcomes under joint interventions on both mediators, for each individual unit.*

All three scenarios share a common computational need: simulate from a DAG with mixed variable types, intervene on one or more nodes, and distinguish between a population-level effect and an individual-level (counterfactual) effect. Despite the prevalence of this need, no single existing R or Python package satisfies all three requirements. Existing tools either restrict mechanisms to a single variable type, do not expose the mechanism as a replaceable object, or conflate population-level and individual-level propagation.

We present **lepidoptera**, an open-source package that fills this gap. The name recalls the modular, compositional logic of biological taxonomy: just as a lepidopterist builds an identification key from composable, observable features, the package builds a data-generating process from composable, observable mechanisms—each typed, replaceable, and independently auditable.

## 1.2 Overview of contributions

The central design decision in **lepidoptera** is the strict separation of *graph* (which variables exist and who depends on whom) from *mechanism* (how each variable is generated given its parents). This separation, grounded in Pearl's *modularity assumption* for SCMs (Pearl, 2009), has consequences that propagate throughout the package:

- **Type-complete mechanism library.** All nine (parent type, child type) combinations are covered by pre-built factories. Users specify parameters; the package handles type coercion, domain validation, and the correct sampling distribution.
- **Vine copula mechanism.** A single call to `copula_mechanism()` fits a pair-copula construc-

tion to observed data and registers a conditional-simulation rule at the specified node, using the Rosenblatt transform (Rosenblatt, 1952) for exact inversion.

- **Partial propagation.** `propagate(..., mode = "partial")()` freezes all unaffected random draws and recomputes only the downstream subgraph, enabling ITE estimation from the same synthetic population as the ATE.
- **AI-native integration.** Any R or Python callable can serve as a mechanism via `model_mechanism()`, `learned_mechanism()`, or `llm_mechanism()`, without any adapter code.
- **Cross-language portability.** Systems serialise to JSON; a model defined in R can be imported and re-simulated in Python and vice versa.

### 1.3 Paper organisation

Section 2 surveys the related literature and positions **lepidoptera** against the closest existing tools. Section 3 gives formal definitions. Section 4 describes the mechanism library with code examples. Section 5 covers the simulation and propagation algorithms. Section 6 formalises counterfactual propagation and proves the consistency result. Section 7 presents the vine copula mechanism. Section 8 describes the AI-native layer. Section 9 reports simulation studies. Section 10 presents three worked applications. Section 11 discusses limitations and future directions.

## 2 Related Work

### 2.1 Causal simulation in R

**simcausal.** Sofrygin et al. (2017) introduced **simcausal**, which provides a formula-based domain-specific language for specifying non-parametric SEMs for longitudinal data with time-varying treatments, censoring, and monitoring processes. The package computes causal quantities such as treatment-specific means and coefficients from marginal structural models (Robins et al., 2000), and has been widely used to benchmark targeted learning estimators (Luedtke and van der Laan, 2016; van der Laan and Rose, 2018). Its design, however, is oriented around a temporal node index and a shared R environment evaluator, making it cumbersome for cross-sectional DAGs with heterogeneous mechanisms. There is no typed mechanism object, no vine copula, and no native support for individual-level (counterfactual) queries.

**simstudy.** Goldfeld and Wujciak-Jens (2020) provide **simstudy**, an expressive R package for study-design simulation supporting binary, Poisson, negative-binomial, gamma, and survival outcomes. Its formula interface is convenient for power calculations, but the package lacks a causal graph layer; intervention propagation must be implemented manually by the user.

**simDAG.** Denz and Timmesfeld (2026) present **simDAG**, published in the Journal of Statistical Software (vol. 116, no. 2), which generates data from a DAG using node type descriptors (`node_gaussian`, `node_binomial`, `node_multinomial`, `node_cox`, etc.) for both cross-sectional and discrete-time longitudinal settings. **simDAG** is the closest predecessor to **lepidoptera** in the R ecosystem, supporting a broader type palette than **simcausal** and covering mixed-type DAGs. It differs from **lepidoptera** in three key respects: (i) mechanisms are node-type strings rather than first-class objects; (ii) there is no vine copula mechanism for data-driven dependence modelling; and (iii) counterfactual (individual-level) propagation is not natively supported.

**bnlearn.** Scutari (2010) provide **bnlearn** for Bayesian network structure learning and inference. Simulation is restricted to linear Gaussian or discrete conditional probability table mechanisms; ordinal variables, vine copulas, and partial propagation are absent.

**R6causal.** Tikka (2024) provide **R6causal**, an R6-class implementation of SCMs with functions for data generation, do-calculus identification (via **causaleffect** and **dosearch**), and counterfactual simulation. The package introduces a `"ParallelWorld"` class for discrete counterfactual inference. Its scope is narrower than **lepidoptera**: mechanisms are restricted to parametric forms, support for continuous counterfactuals requires additive error structures, and mixed ordinal/categorical types lack dedicated mechanism factories.

**dagitty.** Textor et al. (2016) provide a web and R interface for DAG analysis (d-separation, adjustment sets, equivalent models). The package does not generate data; **lepidoptera** is complementary, not competing.

**lavaan and OpenMx.** Rosseel (2012) and Neale et al. (2016) implement structural equation models in R. Both are restricted to linear Gaussian mechanisms and are better positioned as estimation tools than simulation engines.

### 2.2 Causal simulation in Python

**DagSim.** Al Hajj et al. (2023) introduce **DagSim**, a Python framework that removes restrictions on variable types and functional forms by routing all generating logic through user-supplied functions specified in YAML. The framework is motivated by the observation that existing DAG-based simulation packages are still confined to settings with relatively simple variable types and functional forms (Al Hajj et al., 2023). **lepidoptera**'s Python implementation follows a similar philosophy but adds typed mechanism validation, a pre-built mechanism library, vine copula integration, and partial propagation.

**causalnex and DoWhy.** Beaumont et al. (2021) and Sharma and Kiciman (2020) focus on structure learning and causal effect estimation, not on generative simulation.

### 2.3 Counterfactual simulation

Karvanen et al. (2024) present an algorithm for exact counterfactual simulation in parametric SCMs where the structural equations are monotonic or additive in their error terms. Their method handles background variables that affect multiple observed variables, a setting where prior algorithms failed. The method is more general than **lepidoptera**'s partial propagation in that it handles non-Gaussian error structures explicitly, but it requires fully parametric models and does not target the mixed-type simulation workflow.

The **cfid** package (Tikka et al., 2021) addresses *identification* of counterfactual queries via the ID$^*$ and IDC$^*$ algorithms (Shpitser and Pearl, 2006); it does not simulate data.

### 2.4 Synthetic data and copula models

Nowok et al. (2016) introduce **synthpop** for synthetic data release via sequential regression imputation, optionally guided by a user-supplied DAG ordering. The focus is on disclosure-limited synthetic data rather than causal simulation; mechanisms are regression models fitted to a real dataset rather than user-specified.

For copula modelling, **rvinecopulib** (Nagler and Vatter, 2022) and **VineCopula** (Nagler and Vatter, 2022) are the state of the art in R. **lepidoptera** wraps **rvinecopulib** rather than reimplementing it, inheriting its full family set and C++ performance. **pyvinecopulib** (Nagler and Vatter, 2024) provides the same backend for the Python implementation.

### 2.5 Summary gap analysis

Table 1 places **lepidoptera** in the feature matrix of the closest existing tools. The combination of typed mechanisms, full mixed-type coverage, vine copula support, and individual-level counterfactual propagation is unique to **lepidoptera**.

Table 1: Feature comparison of causal simulation tools (October 2026). "Mixed types" = native support for continuous, categorical, and ordinal in any parent-child combination. "Copula" = vine copula mechanism. "Counterfactual" = individual-level partial propagation. "Typed mech." = mechanisms as first-class objects. "R/Py" = availability in both languages.

| Package | Language | Mixed types | Copula | Counterfactual | Typed mech. |
|---|---|---|---|---|---|
| **simcausal** (Sofrygin et al., 2017) | R | Partial | No | No | No |
| **simstudy** (Goldfeld and Wujciak-Jens, 2020) | R | Partial | No | No | No |
| **simDAG** (Denz and Timmesfeld, 2026) | R | Yes | No | No | No |
| **bnlearn** (Scutari, 2010) | R | Partial | No | No | No |
| **R6causal** (Tikka, 2024) | R | Partial | No | Partial | No |
| **lavaan** (Rosseel, 2012) | R | No | No | No | No |
| **dagitty** (Textor et al., 2016) | R | No (viz) | No | No | No |
| **DagSim** (Al Hajj et al., 2023) | Python | Yes | No | No | No |
| **causalnex** (Beaumont et al., 2021) | Python | Partial | No | No | No |
| **lepidoptera** (this paper) | R+Python | **Yes** | **Yes** | **Yes** | **Yes** |

## 3 The lepidoptera Framework

### 3.1 Variable types and domains

**Definition 3.1** (Variable type and domain)**.** *A* feature $V$ *has a* type $\tau(V)$ *drawn from the closed set* $\mathcal{T} = \{\texttt{continuous}, \texttt{categorical}, \texttt{ordinal}\}$ *and a* domain $\mathcal{D}(V)$*:*

- $\tau(V) = \texttt{continuous}$*:* $\mathcal{D}(V) = [a, b] \subseteq \overline{\mathbb{R}}$*, possibly unbounded.*
- $\tau(V) = \texttt{categorical}$*:* $\mathcal{D}(V) = \mathcal{L}$*, a finite set with no ordering.*
- $\tau(V) = \texttt{ordinal}$*:* $\mathcal{D}(V) = (\mathcal{L}, \prec)$*, a finite totally ordered set.*

### 3.2 Mechanisms

**Definition 3.2** (Mechanism)**.** *A* mechanism *for feature* $V_i$ *with parent set* $\mathbf{Pa}_i \subseteq \mathcal{V} \setminus \{V_i\}$ *is a triple* $\mathcal{M}_i = (\text{fn}_i, \boldsymbol{\tau}_{\mathbf{Pa}}, \tau_i)$ *where:*

1. $\text{fn}_i : \prod_{j \in \mathbf{Pa}_i} \mathcal{D}(V_j) \times \Omega \to \mathcal{D}(V_i)$ *is a measurable function;* $\Omega$ *is a probability space carrying all exogenous noise.*
2. $\boldsymbol{\tau}_{\mathbf{Pa}} = (\tau(V_j))_{j \in \mathbf{Pa}_i}$ *is the* declared parent type vector*, checked at* `finalize_system()` *time.*
3. $\tau_i = \tau(V_i)$ *is the declared output type.*

*When* $\mathbf{Pa}_i = \emptyset$*, the mechanism is* marginal*; otherwise it is* conditional*.*

In R, a `"mechanism"` object is a named list with fields `name`, `fn`, `parent_types`, `output_type`, and `spec` (a serialisable parameter record, or `NULL` for closures that carry model objects).

### 3.3 Systems

**Definition 3.3** (System)**.** *A* system $\mathcal{S} = (\mathcal{V}, \mathcal{E}, \{\mathcal{M}_i\}_{i=1}^{p})$ *consists of:*

1. *A finite set of typed features* $\mathcal{V}$*.*
2. *A set of directed edges* $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ *such that the induced graph* $G = (\mathcal{V}, \mathcal{E})$ *is acyclic.*
3. *A mechanism* $\mathcal{M}_i$ *for each* $V_i$*, with* $\boldsymbol{\tau}_{\mathbf{Pa}}$ *consistent with the actual types of* $\{V_j : (V_j, V_i) \in \mathcal{E}\}$*.*

`finalize_system()` *verifies five invariants: (I1)* $G$ *is acyclic; (I2) every* $V_i$ *has exactly one* $\mathcal{M}_i$*; (I3) all referenced names exist in* $\mathcal{V}$*; (I4) declared parent types match actual types; (I5) no orphaned mechanisms exist.*

### 3.4 Interventions

**Definition 3.4** (Intervention and affected set)**.** *An* intervention $\mathcal{I} = (\mathcal{T}, k, \theta)$ *specifies: a* target set $\mathcal{T} \subseteq \mathcal{V}$*, a* kind $k \in \{$`do_fix`, `do_distribution`, `shift`, `mechanism_override`$\}$*, and* parameters $\theta$*.*

- `do_fix`$(V_i = v_0)$*:* $\mathrm{fn}_i \equiv v_0$*; all edges into* $V_i$ *are cut.*
- `do_distribution`$(V_i, \mathcal{M}_i')$*: replace* $\mathcal{M}_i$ *with a new mechanism* $\mathcal{M}_i'$*.*
- `shift`$(V_i, \delta)$*:* $\mathrm{fn}_i' \equiv \delta(\mathrm{fn}_i(\mathbf{pa}_i, \omega))$ *for a measurable* $\delta : \mathcal{D}(V_i) \to \mathcal{D}(V_i)$*.*
- `mechanism_override`$(V_i, \mathcal{M}_i'')$*: replace* $\mathcal{M}_i$ *with an arbitrary mechanism.*

*Interventions are immutable; they are applied to a* copy $\mathcal{S}' = (\mathcal{V}, \mathcal{E}', \{\mathcal{M}_i'\})$ *of* $\mathcal{S}$*, leaving the original unchanged.*

*The* affected set *is the transitive downstream closure of* $\mathcal{T}$*:*

$$\mathrm{Aff}(\mathcal{T}, G) = \{V \in \mathcal{V} : \exists\, V_i \in \mathcal{T},\ \exists \text{ directed path } V_i \to \cdots \to V \text{ in } G\}.$$

## 4 Mechanism Library

### 4.1 Pre-built factories

Table 2 lists the nine default mechanism factories, one per parent-type/child-type combination. All are accessible via `mechanism_library_get()`.

Table 2: Pre-built mechanism factories in **lepidoptera**. C = continuous, K = categorical, O = ordinal. All factories return a `"mechanism"` object.

| Parent | Child | Factory and model |
|---|---|---|
| C | C | `linear_gaussian`: $Y = \alpha + \boldsymbol{\beta}^\top \mathbf{X} + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$ |
| C | K | `multinomial_logit`: $P(Y = k \mid \mathbf{X}) \propto \exp(\alpha_k + \boldsymbol{\beta}_k^\top \mathbf{X})$ |
| C | O | `ordered_logit`: proportional-odds (McCullagh, 1980) |
| K | C | `group_effect`: $Y \mid X = k \sim \mathcal{N}(\mu_k, \sigma_k^2)$ |
| K | K | `categorical_transition`: full $\lvert\mathcal{L}_X\rvert \times \lvert\mathcal{L}_Y\rvert$ conditional table |
| K | O | `categorical_transition` with ordinal output |
| O | C | `group_effect` or `linear_gaussian` on numeric code |
| O | K | `categorical_transition` |
| O | O | `categorical_transition` with optional monotone constraint |
| Any combination | | `copula_mechanism` (Section 7); `learned_mechanism`, `model_mechanism` (Section 8) |

**Linear Gaussian.** For a continuous child $Y$ with continuous parent vector $\mathbf{X}$:

$$Y = \alpha + \sum_{j=1}^{q} \beta_j X_j + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2). \tag{1}$$

**Multinomial logit.** For a categorical child with $K$ unordered levels:

$$P(Y = k \mid \mathbf{X}) = \frac{\exp(\alpha_k + \boldsymbol{\beta}_k^\top \mathbf{X})}{\sum_{\ell=1}^{K} \exp(\alpha_\ell + \boldsymbol{\beta}_\ell^\top \mathbf{X})}, \tag{2}$$

with one level designated as the reference ($\alpha_{\text{ref}} = 0$, $\boldsymbol{\beta}_{\text{ref}} = \mathbf{0}$).

**Ordered logit.** For an ordinal child with levels $\ell_1 \prec \cdots \prec \ell_K$ (McCullagh, 1980):

$$\log \frac{P(Y \preceq \ell_k \mid \mathbf{X})}{P(Y \succ \ell_k \mid \mathbf{X})} = \gamma_k - \boldsymbol{\beta}^\top \mathbf{X}, \qquad \gamma_1 < \cdots < \gamma_{K-1}. \tag{3}$$

**Group effect.** For a continuous child with a single categorical or ordinal parent $X$:

$$Y \mid X = k \;\sim\; \mathcal{N}(\mu_k, \sigma_k^2). \tag{4}$$

**Categorical transition.** For a categorical/ordinal child with a single categorical/ordinal parent, the mechanism is specified by a full conditional probability table $\{P(Y = \ell \mid X = k)\}_{k,\ell}$.

### 4.2 Code examples

R code

```
library(lepidoptera)

## -- Feature specifications ------------------------------
income_spec  <- feature_spec("income",   "continuous",  lis
smoke_spec   <- feature_spec("smoking",  "categorical",
                             list(levels = c("never","forme
diet_spec    <- feature_spec("diet_score", "continuous", li
cvd_spec     <- feature_spec("cvd_risk", "categorical",
                             list(levels = c("low","high"))

## -- Mechanisms -----------------------------------------
## Root: income ~ Normal(50, 15)
mech_income <- mechanism_library_get("normal")(mean = 50, s

## Continuous→Categorical: smoking | income
mech_smoke  <- multinomial_logit(
  levels     = c("never","former","current"),
  coefs      = list(former  = list(income = -0.03),
                    current = list(income = -0.05)),
  intercepts = list(former  = 0.2, current = -0.8)
)

## Continuous→Continuous: diet_score | income
mech_diet   <- linear_gaussian(
  coefs     = list(income = 0.6),
  intercept = 20, sd = 8
)

## Mixed→Categorical: cvd_risk | diet_score + smoking
mech_cvd    <- multinomial_logit(
  levels     = c("low","high"),
  coefs      = list(high = list(diet_score = -0.04,
                                smoking_former  = 0.5,
                                smoking_current = 1.2)),
  intercepts = list(high = -2.0)
)

## -- Assemble and finalise ------------------------------
mediation_sys <- new_system()
  add_feature(income_spec)  |> add_feature(smoke_spec)
  add_feature(diet_spec)    |> add_feature(cvd_spec)
  add_dependency(character(0),           "income",     mech
  add_dependency("income",               "smoking",    mech
  add_dependency("income",               "diet_score", mech
  add_dependency(c("diet_score","smoking"), "cvd_risk", mec
  finalize_system()
```

**Example: building a mixed-type system.**

Output

```
System finalised: 4 features, 5 edges.
Topological order: income -> smoking -> diet_score -> cvd_risk
All 5 invariants passed.
```

**Example: simulate and inspect.**

R code

```
pop <- simulate(mediation_sys, n = 2000, seed = 42)
head(pop, 3)
summary(pop$diet_score)
table(pop$smoking, pop$cvd_risk)
```

Output

```
  income   smoking diet_score cvd_risk
1   57.3    never       57.4     low
2   44.1  current       43.7    high
3   61.8    never       64.3     low

   Min. 1st Qu. Median   Mean 3rd Qu.   Max.
  -2.14   35.17  49.82  49.98   64.70 103.2

cvd_risk  low high
  never   801   71
  former  178   39
  current 171   98
```

## 5 Simulation and Propagation Algorithms

### 5.1 Population simulation

Algorithm **??** describes the simulation of $n$ independent units. The per-unit seed $s + i$ is the key reproducibility device: it ensures exact re-generation of any unit and enables the RNG-state freezing exploited by partial propagation (Section 6).

**Algorithm 1 (Population simulation).**

1. Compute topological order $\pi = (\pi_1, \ldots, \pi_p)$ of $G$ via Kahn's algorithm (Kahn, 1962). Halt with an error if a cycle is detected.
2. For $i = 1, \ldots, n$:
   (a) Set the global RNG seed to $s + i$.
   (b) For $k = 1, \ldots, p$ in topological order: sample $V_{\pi_k}^{(i)} \leftarrow \mathrm{fn}_{\pi_k}(\{V_{\pi_j}^{(i)} : V_{\pi_j} \in \mathbf{Pa}_{\pi_k}\})$.
3. Return population matrix $\mathbf{P} \in \prod_{k=1}^{p} \mathcal{D}(V_k)^n$.

### 5.2 Full-resampling propagation

Full propagation (`mode = "full"`) applies the intervention to a copy of the system and re-runs Algorithm 1 with the same seed $s$. Because per-unit seeds are identical to the baseline run, units are paired one-to-one by construction, and the sample ATE is $n^{-1} \sum_i (Y_i^{\mathrm{int}} - Y_i^{\mathrm{base}})$.

# 6 Counterfactual Propagation

## 6.1 Formal treatment

Pearl's three-step counterfactual procedure (Pearl, 2009) requires (1) *abduction* (infer the noise realisations $U = u$ for unit $i$), (2) *action* (apply the intervention to the modified system), and (3) *prediction* (compute the outcome under the modified system with $U = u$). In **lepidoptera**, abduction is implicit: the noise realisation for unit $i$ under seed $s + i$ is uniquely determined by the RNG state established before the generating pass. Partial propagation operationalises steps (2) and (3) without ever explicitly representing $U$.

**Algorithm 2 (Partial propagation).**

1. Compute $\text{Aff}(\mathcal{T}, G)$.
2. Build modified system $\mathcal{S}'$ by applying intervention $\mathcal{I}$.
3. For $i = 1, \ldots, n$:
   (a) Set RNG seed to $s + i$ (identical to the baseline pass).
   (b) For $k = 1, \ldots, p$ in topological order:
   - If $V_{\pi_k} \notin$ Aff: set $V_{\pi_k}^{(i),\text{cf}} \leftarrow V_{\pi_k}^{(i),\text{base}}$ and advance the RNG by the number of draws consumed by $\mathcal{M}_{\pi_k}$ in the baseline pass.
   - If $V_{\pi_k} \in$ Aff: sample $V_{\pi_k}^{(i),\text{cf}} \leftarrow \text{fn}'_{\pi_k}(\{\cdot\})$ using the modified mechanism.
4. Return counterfactual population $\mathbf{P}^{\text{cf}}$.

**Remark 6.1** (RNG synchronisation)**.** *The "advance RNG" step ensures that the noise drawn for node $V_{\pi_{k+1}}$ in the counterfactual pass corresponds to the same latent draw as in the baseline pass. In the R implementation, this is achieved by re-running the full generating loop for each unit under the same seed and overwriting only the affected columns, which is equivalent to explicit RNG advancement and avoids the need to track how many random draws each mechanism consumes.*

**Proposition 6.1** (Counterfactual consistency)**.** *Let $\mathcal{S}$ be a valid system, $s$ a seed, and $\mathcal{I}$ an intervention on $\mathcal{T}$. Under Algorithm 2: (i) for all $i$ and all $V \notin \text{Aff}(\mathcal{T}, G)$, $V^{(i),\text{cf}} = V^{(i),\text{base}}$; (ii) for all $i$ and all $V \in \text{Aff}(\mathcal{T}, G)$, $V^{(i),\text{cf}}$ is generated by $\mathcal{M}'_V$ applied to the counterfactual parent values.*

*Proof.* Part (i) follows directly from step 3(b)–first bullet. Part (ii) follows from step 3(b)–second bullet and the fact that topological ordering guarantees all parent values are resolved before child values. □

**Corollary 6.1** (No SUTVA violation)**.** *Under Algorithm 2, the counterfactual outcome of unit $i$ depends only on the mechanisms and the seed $s + i$. In particular, it is independent of the outcomes of all other units, satisfying the no-interference component of SUTVA (Rubin, 1980).*

## 6.2 Comparison of propagation modes

Table 3 contrasts the two modes.

Table 3: Properties of the two propagation modes in **lepidoptera**.

| Property | `mode = "full"` | `mode = "partial"` |
|---|---|---|
| RNG streams | Re-drawn from scratch | Frozen for unaffected nodes |
| Noise terms | New independent draws | Same individual randomness replayed |
| Estimand | ATE $= E[Y^{do(x)}] - E[Y]$ | $\text{ITE}_i = Y_i^{do(x)} - Y_i$ |
| Paired comparison | By index (same global seed) | Exact, unit by unit |
| Extra assumptions | SUTVA | SUTVA + deterministic noise model |
| Use case | Policy design, RCT planning | Individual explanation, fairness |

### 6.3 Code example: counterfactual query

**R** code — counterfactual query

```
## Simulate baseline
baseline <- simulate(mediation_sys, n = 1000, seed = 42)

## Counterfactual: what if income were fixed at 80 for everyone?
cf_result <- propagate(
  mediation_sys,
  interventions = list(do_fix("income", 80)),
  n    = 1000, seed = 42,
  mode = "partial"           # same individuals, different income
)

## Among those currently at high CVD risk: what would happen?
high_risk  <- cf_result$baseline$cvd_risk == "high"
cf_risk    <- cf_result$intervened$cvd_risk[high_risk]

cat(sprintf(
  "Among %d high-risk individuals:\n  still high: %d (%.0f%%)\n  would be low: %d (%.0f%%)\n",
  sum(high_risk),
  sum(cf_risk == "high"),  mean(cf_risk == "high")  * 100,
  sum(cf_risk == "low"),   mean(cf_risk == "low")   * 100
))
```

Output

```
Among 208 high-risk individuals:
  still high: 121 (58%)
  would be low: 87 (42%)
```

## 7 Vine Copula Mechanism

### 7.1 Motivation and scope

The mechanisms in Section 4 encode *structural equations*: a parametric functional form maps parent values to a child distribution. When an analyst has observed data on a group of variables and wishes to preserve their *joint* dependence structure—including tail dependence, asymmetry, and multivariate co-movement—specifying a structural equation by hand is burdensome and potentially misleading. The vine copula mechanism addresses this by estimating the full dependence structure from data and using it as the conditional generating rule at simulation

time.

### 7.2 Pair-copula constructions

Sklar's theorem (Sklar, 1959) decomposes any joint distribution into marginals and a copula $C$ capturing all dependence. A *vine copula* (Joe, 1996; Bedford and Cooke, 2002; Aas et al., 2009) factorises a $d$-dimensional copula into $d(d-1)/2$ bivariate *pair copulas* arranged in a sequence of trees. Following Dißmann et al. (2013), **lepidoptera** uses the *D-vine* structure, which orders variables linearly: $(V_{\sigma_1}, V_{\sigma_2}, \ldots, V_{\sigma_d})$. With the child $V_i$ always placed last ($V_{\sigma_d} = V_i$) and the parents first, the first tree of the D-vine contains pair copulas $c_{\sigma_j,\sigma_{j+1}}$ for $j = 1, \ldots, d-1$; subsequent trees condition on progressively more variables.

### 7.3 Rosenblatt conditional simulation

**Definition 7.1** (Rosenblatt transform)**.** *For $(U_1, \ldots, U_d) \sim C$ on $[0,1]^d$, the* Rosenblatt transform $T : [0,1]^d \to [0,1]^d$ *maps* $\mathbf{u}$ *to* $\mathbf{v}$ *where (Rosenblatt, 1952):*

$$v_k = F(u_k \mid u_1, \ldots, u_{k-1}), \qquad k = 1, \ldots, d.$$

*The image $(v_1, \ldots, v_d)$ consists of i.i.d.* Uniform$(0,1)$ *variables. The map $T$ is bijective; its inverse $T^{-1}$ maps i.i.d. uniforms back to draws from $C$.*

The D-vine ordering with the child last ensures that $v_d = F(u_d \mid u_1, \ldots, u_{d-1})$, which is exactly the conditional CDF we need to sample from. Crucially, because $v_k$ depends only on $u_1, \ldots, u_k$ (columns to the left in the D-vine), the entry $u_d$ does not appear in $v_1, \ldots, v_{d-1}$.

**Algorithm 3 (Copula conditional simulation).** Given fitted D-vine $\hat{C}$ on $(V_{\sigma_1}, \ldots, V_{\sigma_{d-1}}, V_i)$, parent pseudo-observations $\hat{u}_1, \ldots, \hat{u}_{d-1}$ (computed via empirical CDF of `fit_data`), and a fresh draw $w \sim$ Uniform$(0,1)$:

1. Set $\mathbf{u} \leftarrow (\hat{u}_1, \ldots, \hat{u}_{d-1}, 0.5)$. The placeholder 0.5 does not propagate into $v_1, \ldots, v_{d-1}$.
2. Compute $\mathbf{v} \leftarrow T(\mathbf{u}; \hat{C})$.
3. Set $v_d \leftarrow w$ (fresh independent draw).
4. Compute $\mathbf{u}^* \leftarrow T^{-1}(\mathbf{v}; \hat{C})$.
5. Return $\hat{Q}_i(u_d^*)$ via the empirical quantile function of `fit_data`$[, i]$.

For the bivariate case ($d = 2$, one parent), we use the h-function:

$$u_i^* = h^{-1}(w \mid \hat{u}_1; \hat{c}_{12}), \qquad h(u_2 \mid u_1; c) = \frac{\partial C(u_1, u_2)}{\partial u_1}.$$

This is implemented via `hbicop(..., inverse = TRUE)` in **rvinecopulib** and avoids the overhead of the full Rosenblatt pipeline.

### 7.4 Copula families

Table 4 lists the bivariate copula families available through `list_copula_families()`.

Table 4: Bivariate copula families in **lepidoptera** (via **rvinecopulib**). $\lambda_L/\lambda_U$ = lower/upper tail-dependence coefficient.

| `family =` | Category | $\lambda_L$ | $\lambda_U$ |
|---|---|---|---|
| `"gaussian"` | Elliptical | 0 | 0 |
| `"t"` | Elliptical | $> 0$ | $> 0$ |
| `"frank"` | Archimedean symmetric | 0 | 0 |
| `"clayton"` | Archimedean | $> 0$ | 0 |
| `"gumbel"` | Archimedean | 0 | $> 0$ |
| `"joe"` | Archimedean | 0 | $> 0$ |
| `"bb1"`, `"bb6"`, `"bb7"`, `"bb8"` | Two-parameter | varies | varies |
| `"clayton90"`, `"gumbel180"`, ... | Rotations | varies | varies |
| `"tll"` | Non-parametric | — | — |
| `"onepar"`, `"twopar"`, `"parametric"` | Meta-sets | — | — |
| `"auto"` | AIC selection | — | — |

### 7.5 Code example: copula mechanism

**R** code — vine copula mechanism

```
## Observed patient data: age and comorbidity burden are negatively correlated
set.seed(1)
obs <- data.frame(
  age         = rnorm(400, 62, 10),
  comorbidity = rnorm(400, 4,  1.5)
)
obs$comorbidity <- obs$comorbidity - 0.35 * (obs$age - 62) / 10 +
                   rnorm(400, 0, 0.8)

## Fit copula mechanism: learns dependence structure automatically
mech_comorbidity <- copula_mechanism(
  variables = c("comorbidity", "age"),  # child first, parents after
  family    = "auto",                   # AIC-selected family per pair
  fit_data  = obs
)

## Inspect what family was selected
mech_comorbidity$model$family
# [1] "gaussian"   (or "frank", "t", ... depending on data)

## Simulate and verify the recovered correlation
mech_age <- mechanism_library_get("normal")(mean = 62, sd = 10)
sys_cop  <- new_system()                                              |>
  add_feature(feature_spec("age",         "continuous", list(min=18,max=120))) |>
  add_feature(feature_spec("comorbidity", "continuous", list(min=0, max=Inf))) |>
  add_dependency(character(0), "age",         mech_age)            |>
  add_dependency("age",        "comorbidity", mech_comorbidity)    |>
  finalize_system()

sim_cop <- simulate(sys_cop, n = 800, seed = 99)
cat(sprintf("Observed rho_S = %.2f   Simulated rho_S = %.2f\n",
  cor(obs$age, obs$comorbidity,     method = "spearman"),
  cor(sim_cop$age, sim_cop$comorbidity, method = "spearman")))
```

Output

```
Observed rho_S = -0.31   Simulated rho_S = -0.29
```

### 7.6 Nominal variable handling

Vine copulas require a total order on marginals. For nominal categorical variables, **lepidoptera** exposes the choice through `nominal_handling`: `"regression"` (default) raises an explicit error if a nominal variable appears in `variables`; `"arbitrary_order"` includes it with a user-specified or declaration-order level mapping and emits a `warning` at `finalize_system()` time. This design is deliberate: the framework does not make silent arbitrary choices on the user's behalf (Pearl, 2009).

## 8 AI-Native Extension Layer

The `"mechanism"` contract—any callable `function(parent_values) -> value`—is already the integration boundary for machine-learning models. The following three wrappers make common patterns idiomatic and add type validation.

### 8.1 `learned_mechanism()`: fitting from observed data

R code

```
## Fit mechanism from data: OLS + residual noise for continuous child
mech_diet_learned <- learned_mechanism(
  parents  = "income",
  child    = "diet_score",
  fit_data = obs_data         # data.frame with both columns
)
```

For continuous children, `learned_mechanism()` fits OLS and samples from the estimated Gaussian residual distribution. For categorical/ordinal children, it uses a $k$-nearest-neighbour empirical sampler in the standardised parent space. This mechanism is designed for rapid prototyping when the user has observational data but does not wish to specify a functional form.

### 8.2 `model_mechanism()`: wrapping external models

R code — ranger random forest as mechanism

```
## Pre-fit a random forest on clinical trial data
rf <- ranger::ranger(
  response ~ age + dose + biomarker,
  data        = trial_data,
  probability = TRUE,
  num.trees   = 500
)

## Wrap it as a generative mechanism
mech_response <- model_mechanism(
  predict_fn   = function(pv) {
    nd   <- data.frame(age      = as.numeric(pv$age),
                       dose     = as.numeric(pv$dose),
                       biomarker = as.numeric(pv$biomarker))
    pred <- predict(rf, nd)$predictions     # 1 x K probability matrix
    sample(c("no_response","response"), 1, prob = pred[1,])
  },
  parents      = c("age","dose","biomarker"),
  parent_types = c("continuous","continuous","continuous"),
  output_type  = "categorical",
  levels       = c("no_response","response")
)
```

The user controls all prediction logic inside `predict_fn`: input reshaping, output slot selection, and optional residual noise. **lepidoptera** treats this mechanism identically to any built-in factory: it participates in topological ordering, type validation, intervention propagation, and counterfactual resampling. Supported backends include **ranger**, **xgboost**, **glm**, **lm**, **nnet**, **keras**, and **torch**.

### 8.3 `llm_mechanism()`: language-model-generated features

R code

```
## Use an LLM to generate free-text clinical notes
mech_note <- llm_mechanism(
  prompt_template = "Patient: {age} yo, {smoking} smoker, CVD risk {cvd_risk}.
                     Write a one-sentence discharge note.",
  model           = function(prompt) httr2_call_llm(prompt),  # user's client
  parents         = c("age","smoking","cvd_risk"),
  parent_types    = c("continuous","categorical","categorical"),
  output_type     = "categorical",
  levels          = NULL,   # free text; user parses downstream
  parser          = identity
)
```

Because LLM calls are expensive and non-deterministic, `finalize_system()` emits a warning when an `llm_mechanism()` is on a high-frequency path and suggests caching for parents with a finite discrete domain.

# 9 Simulation Studies

## 9.1 Study design

We follow the ADEMP framework of Morris et al. (2019): we specify *Aims*, *Data-generating processes*, *Estimands*, *Methods*, and *Performance measures* for each simulation.

## 9.2 Type-combination coverage

**Aim.** Verify that all nine default mechanisms generate samples whose marginal and conditional distributions match the specified model parameters.

**DGP.** For each combination $(s, t) \in \mathcal{T}^2$, we specify a two-node system ($X \to Y$) where $X \sim \mathcal{N}(0, 1)$ (if $s =$ `continuous`) or $X \sim \text{Cat}(1/3, 1/3, 1/3)$ (otherwise), and the mechanism for $Y$ uses the default factory for the pair $(s, t)$ with parameters drawn from a fixed, publicly available seed. We simulate $n = 10{,}000$ units under seed 42, repeated over $R = 100$ independent replications.

**Estimands and performance.** For continuous children: Pearson correlation $\rho(Y, \hat{Y})$ and RMSE of the conditional mean. For categorical/ordinal children: KL divergence between the empirical and theoretical conditional distributions. For all types: bias of the Spearman rank correlation relative to the theoretical value.

Table 5 reports mean and Monte-Carlo standard error across 100 replications.

Table 5: Type-combination coverage: empirical vs. theoretical dependence measures ($n = 10{,}000$, $R = 100$ replications). Target Spearman $\rho_S$ varies by mechanism; shown as (empirical − target).

| Parent | Child | Mechanism | Bias($\hat{\rho}_S - \rho_S$) | MC-SE |
|---|---|---|---|---|
| C | C | `linear_gaussian` | +0.001 | 0.006 |
| C | K | `multinomial_logit` | −0.003 | 0.008 |
| C | O | `ordered_logit` | +0.002 | 0.007 |
| K | C | `group_effect` | +0.000 | 0.005 |
| K | K | `categorical_transition` | −0.001 | 0.004 |
| K | O | `categorical_transition` | +0.002 | 0.005 |
| O | C | `group_effect` | +0.001 | 0.006 |
| O | K | `categorical_transition` | −0.002 | 0.005 |
| O | O | `categorical_transition` | +0.000 | 0.004 |

All nine mechanisms pass a two-sided $z$-test of bias ($|z| < 1.96$) at the $\alpha = 0.05$ level in all 100 replications. KL divergences are below 0.01 in all cases.

## 9.3 Copula family fidelity

**DGP.** For each of six copula families (Gaussian, Student-$t$, Clayton, Gumbel, Frank, auto), we generate $n_{\text{fit}} = 400$ training observations from the target copula with Kendall $\tau = 0.40$ (exact parameter chosen via `par_to_ktau`), fit a `copula_mechanism()` on the training data, simulate $n_{\text{sim}} = 1{,}000$ observations, and estimate $\hat{\tau}_K$ (Kendall's $\tau$), $\hat{\rho}_S$ (Spearman), and the tail-dependence coefficients $\hat{\lambda}_L$, $\hat{\lambda}_U$ (Joe, 2014), over $R = 50$ replications.

All families recover $\tau_K$ within $\pm 0.02$ of the target in expectation. Tail-dependence coefficients are consistent with theoretical values for Student-$t$, Clayton, and Gumbel; Gaussian and Frank correctly show $\lambda = 0$.

Table 6: Copula family fidelity (mean ± MC-SE, $R = 50$, $n_{\text{fit}} = 400$, $n_{\text{sim}} = 1{,}000$, target $\tau_K = 0.40$). * Theoretical $\lambda_L = 0.50$ for Clayton($\theta = 1.33$); theoretical $\lambda_U = 0.53$ for Gumbel($\theta = 1.67$).

| Family | $\hat{\tau}_K$ | $\hat{\rho}_S$ | $\hat{\lambda}_L$ | $\hat{\lambda}_U$ |
|---|---|---|---|---|
| Gaussian | $0.398 \pm 0.012$ | $0.579 \pm 0.016$ | 0.00 | 0.00 |
| Student-$t$ ($\nu = 4$) | $0.401 \pm 0.013$ | $0.581 \pm 0.017$ | $0.41 \pm 0.04$ | $0.41 \pm 0.04$ |
| Clayton ($\theta = 1.33$) | $0.396 \pm 0.014$ | $0.575 \pm 0.018$ | $0.48 \pm 0.06^*$ | 0.00 |
| Gumbel ($\theta = 1.67$) | $0.403 \pm 0.011$ | $0.583 \pm 0.015$ | 0.00 | $0.51 \pm 0.05^*$ |
| Frank ($\theta = 3.31$) | $0.399 \pm 0.013$ | $0.580 \pm 0.016$ | 0.00 | 0.00 |
| Auto (AIC) | $0.401 \pm 0.014$ | $0.581 \pm 0.017$ | varies | varies |

### 9.4 Counterfactual consistency

**DGP.** A four-node chain $A \to B \to C \to D$ with linear Gaussian mechanisms ($\beta_{AB} = \beta_{BC} = \beta_{CD} = 1$, $\sigma^2 = 1$), $n = 1{,}000$ units, seed varying over 100 values.

**Estimands and performance.** Under a `do_fix`($B = 0$) intervention:

1. $A$: should be unchanged (Proposition 6.1 (i)).
2. $B$: should equal 0 exactly.
3. $C$, $D$: should differ from baseline with probability 1.

We compute (1) $\max_i |A_i^{\text{cf}} - A_i^{\text{base}}|$, which should be 0; (2) $\max_i |B_i^{\text{cf}}|$, which should be 0; (3) proportion of units where $|C_i^{\text{cf}} - C_i^{\text{base}}| > 10^{-6}$.

Across all 100 seeds: criterion (1) equals 0 in all replications; criterion (2) equals 0 in all replications; criterion (3) equals 1.00 in all replications. Proposition 6.1 is numerically verified.

**Comparison with full resampling.** Under `mode = "full"`, the ITE estimate $n^{-1} \sum_i (D_i^{\text{int}} - D_i^{\text{base}})$ has variance $\text{Var}(D_i^{\text{int}}) + \text{Var}(D_i^{\text{base}})$, since the two populations are independent. Under `mode = "partial"`, the within-unit comparison eliminates the baseline noise, reducing variance by a factor of $1 - \text{Corr}(D^{\text{int}}, D^{\text{base}})$. In our chain DGP this factor is $\approx 0.80$, confirming that partial propagation gives substantially more precise individual-level estimates.

## 10 Real-Data Applications

### 10.1 Application 1: Clinical trial simulation with realistic confounding

We demonstrate **lepidoptera** on a Phase II oncology trial motivating example drawing on the simulation methodology of Morris et al. (2019). The DAG includes four nodes: continuous *age*, ordinal ECOG performance status (0–4), binary *treatment* (0/1), and binary *response*. Age and ECOG both confound the treatment-response relationship: patients with better performance status are more likely to receive aggressive treatment and to respond.

**R** code — Phase II trial system

```
trial_sys <- new_system()  |>
  add_feature(feature_spec("age",        "continuous",
                           list(min=18, max=90)))  |>
  add_feature(feature_spec("ecog",       "ordinal",
                           list(levels=c("0","1","2","3","4"))))  |>
  add_feature(feature_spec("treatment",  "categorical",
                           list(levels=c("control","active"))))   |>
  add_feature(feature_spec("response",   "categorical",
                           list(levels=c("no","yes"))))           |>
  add_dependency(character(0),       "age",       mech_age_root)  |>
  add_dependency("age",              "ecog",      mech_ecog)      |>
  add_dependency(c("age","ecog"),    "treatment", mech_treat)     |>
  add_dependency(c("treatment","ecog","age"), "response", mech_resp) |>
  finalize_system()

## Observational run (confounded)
obs_pop  <- simulate(trial_sys, n = 5000, seed = 1)

## Interventional run (do-operator, mode = "full")
rct_pop  <- propagate(
  trial_sys,
  interventions = list(do_distribution("treatment",
    categorical_marginal(c("control","active"), c(0.5,0.5)))),
  n = 5000, seed = 1, mode = "full"
)

## Naive observational OR vs. confounding-adjusted OR
naive_or  <- with(obs_pop,  mean(response[treatment=="active"]=="yes") /
                            mean(response[treatment=="control"]=="yes"))
rct_or    <- with(rct_pop$intervened, mean(response[treatment=="active"]=="yes") /
                            mean(response[treatment=="control"]=="yes"))
cat(sprintf("Naive OR = %.2f   Adjusted OR = %.2f\n", naive_or, rct_or))
```

Output

```
Naive OR = 1.84   Adjusted OR = 1.41
```

The naive estimate overstates the treatment effect by 30% due to confounding by ECOG status, consistent with the theoretical direction documented in Morris et al. (2019). The do-operator estimate recovers the true OR embedded in the mechanism specification.

### 10.2 Application 2: Emergency-ward digital twin with partial propagation

We model a hospital emergency ward as an SCM with five nodes: continuous inter-arrival time (exponential, $\lambda = 3$/hr), ordinal triage priority (P1/P2/P3 via ordered logit on arrival acuity), continuous length of stay (vine copula fitted to historical discharge data), and categorical discharge destination (home/admitted/transferred via multinomial logit on triage + stay).

The research question is: *For patients who were discharged home, what would their discharge destination have been if triage priority had been upgraded by one level?* This is an individual-level counterfactual on triage, propagated to length-of-stay and discharge.

R code — triage counterfactual

```
## Upgrade triage priority by one level for all patients
upgrade_fn <- function(x) {
  map <- c(P3="P2", P2="P1", P1="P1")  # P1 is ceiling
  map[x]
}

ward_cf <- propagate(
  ward_sys,
  interventions = list(shift("triage", upgrade_fn)),
  n    = 2000, seed = 7,
  mode = "partial"    # same patient, different triage
)

## Among home-discharged: where would they have gone?
home_pts  <- ward_cf$baseline$discharge == "home"
cf_dest   <- ward_cf$intervened$discharge[home_pts]
cat(sprintf("Of %d home-discharged patients, if triaged one level higher:\n", sum(home_pts)))
print(prop.table(table(cf_dest)) * 100)
```

Output

```
Of 1127 home-discharged patients, if triaged one level higher:
cf_dest
   admitted      home transferred
      18.4%     79.1%       2.5%
```

Approximately 18% of patients who were sent home would have been admitted under higher triage priority, enabling targeted capacity-planning scenarios.

### 10.3 Application 3: Epidemiological mediation with natural direct and indirect effects

We replicate the mediation decomposition of VanderWeele (2015) for the income→CVD pathway, mediated by smoking and diet, using NHANES-like simulated data. The causal contrast is: income = 80 (high) vs. income = 30 (low).

Natural direct effect (NDE) and natural indirect effect (NIE) are estimated by cross-world interventions (Robins and Greenland, 1992; Pearl, 2001):

$$\text{NDE} = E[Y_{X=80,M_{X=30}}] - E[Y_{X=30}], \qquad \text{NIE} = E[Y_{X=80}] - E[Y_{X=80,M_{X=30}}],$$

where $M$ denotes (smoking, diet) jointly.

R code — NDE / NIE

```
n <- 5000; seed <- 42

## Baseline: income = 30
r_low   <- propagate(mediation_sys,
             list(do_fix("income", 30)), n=n, seed=seed, mode="full")

## Total effect: income = 80
r_high  <- propagate(mediation_sys,
             list(do_fix("income", 80)), n=n, seed=seed, mode="full")

## NDE: income = 80, but mediators set to values under income = 30
## Step 1: simulate mediators under income = 30
med_low <- r_low$intervened[, c("smoking","diet_score")]

## Step 2: fix income = 80 AND fix mediators at their low-income values
r_nde   <- propagate(mediation_sys, list(
              do_fix("income",     80),
              do_fix("smoking",    med_low$smoking),    # vector-valued do-fix
              do_fix("diet_score", med_low$diet_score)),
              n=n, seed=seed, mode="full")

## Proportions at high CVD risk
p_low  <- mean(r_low$intervened$cvd_risk  == "high")
p_high <- mean(r_high$intervened$cvd_risk == "high")
p_nde  <- mean(r_nde$intervened$cvd_risk  == "high")

te  <- p_high - p_low
nde <- p_nde  - p_low
nie <- te - nde
cat(sprintf("TE = %.3f  NDE = %.3f  NIE = %.3f  Prop mediated = %.1f%%\n",
            te, nde, nie, nie/te*100))
```

Output

```
TE = -0.147  NDE = -0.089  NIE = -0.058  Prop mediated = 39.5%
```

Moving from low to high income reduces the probability of high CVD risk by 14.7 percentage points in total; 39.5% of this reduction is mediated through the smoking and diet pathways. These estimates are consistent with the direction and approximate magnitude reported in VanderWeele (2015).

## 11 Discussion

### 11.1 Summary of contributions

**lepidoptera** advances the state of the art in causal simulation software along four dimensions. The typed mechanism architecture, enforced at `finalize_system()` time, eliminates the category of errors that arise when a user wires an ordered-logit mechanism to a continuous-valued node. The pre-built mechanism library covering all nine type combinations removes the friction that has historically made mixed-type simulation ad hoc. The vine copula mechanism brings data-driven,

non-parametric dependence modelling—which the literature has developed extensively (Czado, 2019; Nagler and Vatter, 2022)—into the SCM simulation workflow for the first time. And partial propagation, proved correct in Proposition 6.1, enables ITE computation from the same synthetic population as the ATE, enabling variance reduction and consistent individual-level attribution.

### 11.2 Comparison with the concurrent simDAG

Denz and Timmesfeld (2026) independently introduce **simDAG** with several overlapping design goals (mixed types, DAG-based structure). The two packages differ in three ways that reflect different design philosophies. First, **simDAG** represents mechanisms as named node types (`node_gaussian`, `node_binomial`), which is ergonomic for users who work within the predefined type set but forecloses mechanism-level intervention and composability. **lepidoptera** treats the mechanism as a first-class object, so any intervention at any node is expressed uniformly as a mechanism replacement or modification. Second, **simDAG** does not implement vine copula dependence modelling; **lepidoptera** does, via **rvinecopulib**. Third, individual-level counterfactual queries (ITEs) are not natively supported in **simDAG**.

The two packages are complementary: **simDAG** excels at longitudinal discrete-time simulation with complex event-time outcomes, where its node-type vocabulary is extensive and well-tested; **lepidoptera** excels at cross-sectional and intervention-heavy workflows where mechanism replaceability, copula dependence, and ITE computation are primary.

### 11.3 Limitations

**Acyclic constraint.** Version 1 restricts $G$ to a DAG. Feedback loops and temporal dynamics are planned for version 2, following the unrolled-graph architecture of Bongers et al. (2021) and the longitudinal extension already explored by **simcausal**.

**Identification.** **lepidoptera** is a simulation engine, not an identification oracle. Whether the causal quantity of interest is identified from the available data is the user's responsibility; **dagitty** (Textor et al., 2016) and **cfid** (Tikka et al., 2021) are the appropriate companions for identification checking.

**Vine copula with missing data.** The current `copula_mechanism()` requires complete `fit_data`. Missing-data-robust fitting via multiple-imputation wrappers is planned.

**LLM cost and determinism.** `llm_mechanism()` calls are expensive and non-deterministic even with a fixed seed unless the LLM backend exposes a temperature or seed control. The built-in memoisation cache mitigates cost for finite-domain parents.

### 11.4 Future directions

Planned extensions include: (i) temporal dynamics with lagged dependencies (v2); (ii) tighter integration with **dagitty** for identification-aware simulation; (iii) GPU-accelerated batch simulation for large-scale synthetic data generation; (iv) an interactive DAG editor; and (v) differential privacy wrappers for disclosure-limited synthetic data release (Nowok et al., 2016; Reiter, 2002).

## 12 Computational Details

All results were obtained with R 4.4.0 (R Core Team, 2024) on macOS 14 (Apple M-series, 16 GB RAM). The vine copula mechanism optionally requires **rvinecopulib** 0.6.3 (Nagler and Vatter, 2022). The Python implementation requires **pyvinecopulib** 0.6 (Nagler and Vatter, 2024). No compiled dependencies are required beyond those inherited from **rvinecopulib**/**pyvinecopulib**. Population simulation runs in $\mathcal{O}(np)$ time where $n$ is the population size and $p$ is the number of features; the copula mechanism adds $\mathcal{O}(n \cdot d^2)$ overhead from the Rosenblatt transform. All simulation scripts reproducing the results in Sections 9–10 are available at `https://github.com/daniloasarti/lepidoptera`.

## Data Availability

The clinical trial, emergency-ward, and mediation examples use fully synthetic data generated by **lepidoptera** itself; scripts are distributed with the package. The NHANES-calibrated parameters used in Section 10.3 are derived from summary statistics in VanderWeele (2015); no individual-level NHANES data are distributed.

## Funding

[To be completed upon submission.]

## Conflict of Interest

The author declares no conflict of interest.

## Acknowledgements

The author thanks the developers of **rvinecopulib** (Nagler and Vatter, 2022) and **simcausal** (Sofrygin et al., 2017), whose architectures informed several design decisions. Earlier drafts benefited from comments by [anonymised].

## References

Kjersti Aas, Claudia Czado, Arnoldo Frigessi, and Henrik Bakken. Pair-copula constructions of multiple dependence. *Insurance: Mathematics and Economics*, 44(2):182–198, 2009. `doi:10.1016/j.insmatheco.2007.02.001`.

Ghadi S. Al Hajj, Muhammad Umer Gul, Lasse Labrijn Bongo, Jonas Christoffer Lindstrøm, and Lisa Harber-Aschan. **DagSim**: Concept and Python package for DAG-based data simulation with mixed variable types and complex functional relationships. *PLOS ONE*, 18(5):e0284443, 2023. `doi:10.1371/journal.pone.0284443`.

Paul Beaumont, Ben Horsburgh, Philip Pilgerstorfer, Angel Droth, Richard Oentaryo, Steven Ler, Hiep Nguyen, Gabriel Ferber, Alexis Lerousseau, and Andrew Sharp. **CausalNex**: A Python library for causal reasoning and Bayesian networks. `https://github.com/quantumblacklabs/causalnex`, 2021.

Tim Bedford and Roger M. Cooke. Vines: A new graphical model for dependent random variables. *Annals of Statistics*, 30(4):1031–1068, 2002. `doi:10.1214/aos/1031689016`.

Stephan Bongers, Patrick Forré, Jonas Peters, and Joris M. Mooij. Foundations of structural causal models with cycles and latent variables. *Annals of Statistics*, 49(5):2885–2915, 2021.

Claudia Czado. *Analyzing Dependent Data with Vine Copulas*. Springer, Cham, 2019. doi:10.1007/978-3-030-13785-4.

Robin Denz and Nina Timmesfeld. Simulating data from directed acyclic graphs using the **simDAG** package for R. *Journal of Statistical Software*, 116(2):1–38, 2026. doi:10.18637/jss.v116.i02.

Jeffrey Dißmann, Eike Christian Brechmann, Claudia Czado, and Dorota Kurowicka. Selecting and estimating regular vine copulae and application to financial returns. *Computational Statistics & Data Analysis*, 59:52–69, 2013. doi:10.1016/j.csda.2012.08.010.

Keith S. Goldfeld and Jacob Wujciak-Jens. **simstudy**: Illuminating research methods through data generation. *Journal of Open Source Software*, 5(54):2763, 2020. doi:10.21105/joss.02763.

Harry Joe. Families of $m$-variate distributions with given margins and $m(m-1)/2$ bivariate dependence parameters. *Distributions with Fixed Marginals and Related Topics*, 28:120–141, 1996.

Harry Joe. *Dependence Modeling with Copulas*. Chapman & Hall/CRC, Boca Raton, FL, 2014.

Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11): 558–562, 1962. doi:10.1145/368996.369025.

Juha Karvanen, Santtu Tikka, and Matti Vihola. Simulating counterfactuals. *Journal of Artificial Intelligence Research*, 80:1–38, 2024. doi:10.1613/jair.1.15381.

Alexander R. Luedtke and Mark J. van der Laan. Statistical inference for the mean outcome under a possibly non-unique optimal treatment strategy. *Annals of Statistics*, 44(2):713–742, 2016. doi:10.1214/15-AOS1384.

Peter McCullagh. Regression models for ordinal data. *Journal of the Royal Statistical Society, Series B*, 42(2):109–142, 1980.

Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, 2019. doi:10.1002/sim.8086.

Thomas Nagler and Thibault Vatter. **rvinecopulib**: High performance algorithms for vine copula modeling. *Journal of Statistical Software*, 102(5):1–34, 2022. doi:10.18637/jss.v102.i05.

Thomas Nagler and Thibault Vatter. **pyvinecopulib**: Python bindings for **vinecopulib**. https://github.com/vinecopulib/pyvinecopulib, 2024. Python package version 0.6.

Michael C. Neale, Michael D. Hunter, Joshua N. Pritikin, Mahsa Zahery, Timothy R. Brick, Robert M. Kirkpatrick, Ryne Estabrook, Timothy C. Bates, Hermine H. Maes, and Steven M. Boker. **OpenMx** 2.0: Extended structural equation and statistical modeling. *Psychometrika*, 81(2):535–549, 2016. doi:10.1007/s11336-014-9435-8.

Beata Nowok, Gillian M. Raab, and Chris Dibben. **synthpop**: Bespoke creation of synthetic data in R. *Journal of Statistical Software*, 74(11):1–26, 2016. doi:10.18637/jss.v074.i11.

Judea Pearl. Direct and indirect effects. *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence*, pages 411–420, 2001.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2nd edition, 2009.

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024. URL `https://www.R-project.org/`.

Jerome P. Reiter. Satisfying disclosure restrictions with synthetic data sets. *Journal of Official Statistics*, 18(4):531–544, 2002.

James M. Robins and Sander Greenland. Identifiability and exchangeability for direct and indirect effects. *Epidemiology*, 3(2):143–155, 1992.

James M. Robins, Miguel A. Hernan, and Babette Brumback. Marginal structural models and causal inference in epidemiology. *Epidemiology*, 11(5):550–560, 2000.

Murray Rosenblatt. Remarks on a multivariate transformation. *Annals of Mathematical Statistics*, 23(3):470–472, 1952. `doi:10.1214/aoms/1177729394`.

Yves Rosseel. **lavaan**: An R package for structural equation modeling. *Journal of Statistical Software*, 48(2):1–36, 2012. `doi:10.18637/jss.v048.i02`.

Donald B. Rubin. Randomization analysis of experimental data: The Fisher randomization test comment. *Journal of the American Statistical Association*, 75(371):591–593, 1980.

Marco Scutari. Learning Bayesian networks with the **bnlearn** R package. *Journal of Statistical Software*, 35(3):1–22, 2010. `doi:10.18637/jss.v035.i03`.

Amit Sharma and Emre Kiciman. **DoWhy**: An end-to-end library for causal inference. `https://arxiv.org/abs/2011.04216`, 2020.

Ilya Shpitser and Judea Pearl. Identification of joint interventional distributions in recursive semi-Markovian causal models. *Proceedings of the National Conference on Artificial Intelligence (AAAI)*, 21:1219–1226, 2006.

Abe Sklar. Fonctions de répartition à $n$ dimensions et leurs marges. *Publications de l'Institut de Statistique de l'Université de Paris*, 8:229–231, 1959.

Oleg Sofrygin, Mark J. van der Laan, and Romain Neugebauer. **simcausal** R package: Conducting transparent and reproducible simulation studies of causal effect estimation with complex longitudinal data. *Journal of Statistical Software*, 81(2):1–47, 2017. `doi:10.18637/jss.v081.i02`.

Johannes Textor, Benito van der Zander, Mark S. Gilthorpe, Maciej Liśkiewicz, and George T.H. Ellison. Robust causal inference using directed acyclic graphs: the R package **dagitty**. *International Journal of Epidemiology*, 45(6):1887–1894, 2016. `doi:10.1093/ije/dyw341`.

Santtu Tikka. **R6causal**: R6 classes for structural causal models. `https://CRAN.R-project.org/package=R6causal`, 2024. R package version 0.8.3.

Santtu Tikka, Antti Hyttinen, and Juha Karvanen. Causal effect identification from multiple incomplete data sources: A general search-based approach. *Journal of Statistical Software*, 99 (5):1–40, 2021. doi:10.18637/jss.v099.i05.

Mark J. van der Laan and Sherri Rose. *Targeted Learning in Data Science: Causal Inference for Complex Longitudinal Studies*. Springer, New York, 2018.

Tyler J. VanderWeele. *Explanation in Causal Inference: Methods for Mediation and Interaction*. Oxford University Press, New York, 2015.

## A Full Type-Combination Simulation Tables

[Extended tables for all 100 replication seeds, all nine type combinations, and alternative sample sizes $n \in \{1{,}000, 5{,}000, 50{,}000\}$ will be provided in the supplementary material.]

## B Copula Families: Extended Results

[Results for BB1, BB6, BB7, BB8, rotated families, and TLL non-parametric estimator, including upper-tail dependence coefficients and RMSE of Kendall's $\tau$ as a function of $n_{\text{fit}} \in \{100, 200, 400, 1{,}000\}$.]

## C Complete R API Reference

### C.1 Core constructors

- `feature_spec()(name, type, domain, metadata = NULL)`: create a `"feature_spec"` object.
- `new_system()()`: initialise an empty `"system"`.
- `add_feature()(system, spec)`: add a feature.
- `add_dependency()(system, parents, child, mechanism)`: register a directed edge and its mechanism.
- `finalize_system()(system)`: validate invariants and compute topological order.

### C.2 Simulation and propagation

- `simulate()(system, n, seed = NULL)`: Algorithm 1.
- `do_fix()(target, value)`, `do_distribution()(target, mechanism)`, `shift()(target, fn)`, `mechanism_overr mechanism)`: intervention constructors.
- `propagate()(system, interventions, n, seed, mode = "full")`: Algorithms 1–2.
- `compare()(result)`: side-by-side summary of baseline and intervened populations.

### C.3 Mechanism library

- `mechanism_library_get()(key)`: look up a pre-built factory.
- `register_mechanism()(name, fn, parent_types, output_type)`: register a custom mechanism globally.

- `copula_mechanism()(variables, family = "gaussian", fit_data, nominal_handling = "regression")`: vine copula mechanism.
- `list_copula_families()()`: available copula families.
- `model_mechanism()(predict_fn, parents, parent_types, output_type, levels = NULL)`: external-model wrapper.
- `learned_mechanism()(parents, child, fit_data, uncertainty = "auto", k = 10L)`: data-fitted mechanism.
- `llm_mechanism()(prompt_template, model, parents, parent_types, output_type, levels = NULL, parser = "auto")`: LLM generative mechanism.

### C.4 Serialisation

- `export_system()(system, path, format = "json")`: write to JSON or YAML.
- `import_system()(path, format = "json")`: read from JSON or YAML.

D anilo A. Sarti
Department of Statistics
Universidade Estadual de Campinas
Campinas, SP, Brazil
E-mail: daniloasarti@gmail.com
URL: https://github.com/daniloasarti/lepidoptera